



Vulnerability Assessment Report for a Live Website

:- BY MEET KHAMAR (FIT/MAY26/CS8556)



Table of Contents

Introduction

Vulnerability Findings

Tools Used

Risk Summary

Conclusion

Introduction

- **Target:** <https://demo.owasp-juice.shop/> || <http://localhost:3000> (OWASP Juice Shop)
- **Objective:** Perform a vulnerability assessment to identify security flaws.
- **Summary:** The assessment revealed several misconfigurations, including exposed directories, missing security headers, and excessive API data exposure. Immediate remediation is recommended to prevent data leaks and client-side attacks.

Vulnerability Findings

Finding 1: Exposed Directory Listing

- **Risk Level:** High
- **Vulnerability Description:** The web application fails to restrict access to sensitive directories. An unauthenticated user can discover and access the [/ftp](#) directory.
- **Business Impact:** Attackers can browse and download files directly from the server. This can lead to the exposure of sensitive business documents, backup files, user data, or source code, resulting in a severe data breach.

```

(meet-kingsman)-[~]
$ dirb http://localhost:3000/

-----
DIRB v2.22
By The Dark Raver
-----

START_TIME: Sat May 23 03:10:24 2026
URL_BASE: http://localhost:3000/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

-----

GENERATED WORDS: 4612

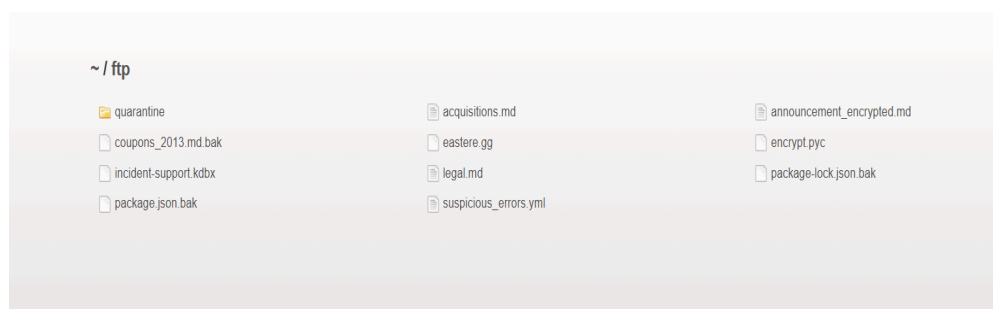
---- Scanning URL: http://localhost:3000/ ----
+ http://localhost:3000/assets (CODE:301|SIZE:156)
+ http://localhost:3000/ftp (CODE:200|SIZE:11317)
+ http://localhost:3000/media (CODE:301|SIZE:155)
+ http://localhost:3000/profile (CODE:500|SIZE:1043)
+ http://localhost:3000/promotion (CODE:200|SIZE:5863)
+ http://localhost:3000/redirect (CODE:500|SIZE:3113)
+ http://localhost:3000/robots.txt (CODE:200|SIZE:28)
+ http://localhost:3000/video (CODE:200|SIZE:10075518)
+ http://localhost:3000/Video (CODE:200|SIZE:10075518)

-----

END_TIME: Sat May 23 03:10:59 2026
DOWNLOADED: 4612 - FOUND: 9

```

- Evidence: (<http://localhost:3000/ftp>)



-
- **Remediation Steps:** 1. Disable directory browsing/indexing on the web server configuration. 2. Implement strict access controls and authentication requirements for the `/ftp` endpoint.

Finding 2: Missing HTTP Security Headers

Risk Level: Medium

Vulnerability Description: The web server is not enforcing essential security headers in its HTTP responses. Specifically, `Content-Security-Policy` (CSP) and `Strict-Transport-Security` (HSTS) are missing.

Business Impact: Without these headers, the application and its users are more susceptible to client-side attacks such as Cross-Site Scripting (XSS), data injection, and man-in-the-middle (MitM) attacks.

Evidence: (`curl -I http://localhost:3000/`)

```
(meet-k@kingsman)-[~]
$ curl -I http://localhost:3000/
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Fri, 22 May 2026 21:36:12 GMT
ETag: W/"26af-19e519e0afc"
Content-Type: text/html; charset=UTF-8
Content-Length: 9903
Vary: Accept-Encoding
Date: Fri, 22 May 2026 21:43:59 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Remediation Steps:

1. Configure the server to include a robust **Content-Security-Policy** header to dictate which dynamic resources are allowed to load.
2. Implement **Strict-Transport-Security** to force all connections over HTTPS.

Finding 3: Excessive API Information Exposure

Risk Level: Medium

Vulnerability Description: The unauthenticated API endpoint (`/api/Products`) returns excessive backend data. While product information is public, the API leaks internal database IDs and exact database creation/modification timestamps (`createdAt`, `updatedAt`).

Business Impact: Exposing internal database structures and timestamps provides attackers with reconnaissance data. It helps them understand the backend architecture, which can be leveraged to craft more sophisticated attacks (like Insecure Direct Object Reference or Mass Assignment).

Evidence: (`curl -s http://localhost:3000/api/Products | jq`)

```
(meet-k@kingsman)-[~]
$ curl -s http://localhost:3000/api/Products | jq
{
  "status": "success",
  "data": [
    {
      "id": 1,
      "name": "Apple Juice (1000ml)",
      "description": "The all-time classic.",
      "price": 1.99,
      "deluxePrice": 0.99,
      "image": "apple_juice.jpg",
      "createdAt": "2026-05-22T21:36:10.436Z",
      "updatedAt": "2026-05-22T21:36:10.436Z",
      "deletedAt": null
    },
    {
      "id": 2,
      "name": "Orange Juice (1000ml)",
      "description": "Made from oranges hand-picked by Uncle Dittmeyer.",
      "price": 2.99,
      "deluxePrice": 2.49,
      "image": "orange_juice.jpg",
      "createdAt": "2026-05-22T21:36:10.436Z",
      "updatedAt": "2026-05-22T21:36:10.436Z",
      "deletedAt": null
    },
    {
      "id": 3,
      "name": "Eggfruit Juice (500ml)",
      "description": "Now with even more exotic flavour.",
      "price": 8.99,
      "deluxePrice": 8.99,
      "image": "eggfruit_juice.jpg",
      "createdAt": "2026-05-22T21:36:10.436Z",
      "updatedAt": "2026-05-22T21:36:10.436Z",
      "deletedAt": null
    }
  ]
}
```

Remediation Steps:

1. Implement Data Transfer Objects (DTOs) on the backend to filter out sensitive or unnecessary fields before sending the JSON payload to the client.
2. Only return the exact fields required by the frontend application (e.g., name, description, price, image).

Finding 4: Missing X-Frame-Options Header

Risk Level: Medium

Vulnerability Description: The application responses do not include the X-Frame-Options HTTP security header. This allows the website to be embedded inside an iframe on a malicious external website.

Business Impact: Attackers can exploit this weakness to perform clickjacking attacks. Users may unknowingly click hidden buttons or perform unintended actions while interacting with a malicious webpage.

Evidence: (OWASP ZAP Alert: *Missing Anti-clickjacking Header*)

The screenshot displays the OWASP ZAP Manual Explore interface. On the left, the 'Contexts' pane shows a tree structure with 'Sites' and 'Assets'. The 'Assets' section is expanded, showing various GET requests for JavaScript files. The main pane is titled 'Manual Explore' and contains a form for launching a browser to explore the application. Below the form, a list of alerts is shown, including 'SQL Injection (2)', 'CSP: Failure to Define Directive with No Fallback (Systemic)', 'CSP: Wildcard Directive (Systemic)', 'CSP: script-src unsafe-eval (8)', 'CSP: script-src unsafe-inline (8)', 'CSP: style-src unsafe-inline (Systemic)', 'Content Security Policy (CSP) Header Not Set (3)', 'Cross-Domain Misconfiguration (Systemic)', 'Information Disclosure - JWT in Browser localStorage', 'Missing Anti-clickjacking Header', and 'Session ID in URL Rewrite (Systemic)'. The 'Missing Anti-clickjacking Header' alert is highlighted, showing details such as 'CWE ID: 89', 'WASC ID: 19', 'Source: Active (40018 - SQL Injection)', 'Input Vector: JSON', and a description stating 'SQL injection may be possible.' The solution provided is 'Do not trust client side input, even if there is client side validation in place. In general, handle all data on the server side.'

Remediation Steps:

Configure the web server to include the X-Frame-Options header with either DENY or SAMEORIGIN policy.

Implement additional clickjacking protection using the Content-Security-Policy frame-ancestors directive.

Finding 5: Cross-Site Scripting (XSS)

Risk Level: High

Vulnerability Description: The application improperly validates user-supplied input, allowing attackers to inject malicious JavaScript code into web pages viewed by other users.

Business Impact: Successful exploitation may allow attackers to steal session cookies, manipulate application content, redirect users to malicious websites, or perform actions on behalf of authenticated users.

Evidence: (OWASP ZAP Alert: Cross Site Scripting (Reflected))

Affected Parameter: search

The screenshot displays the OWASP ZAP Manual Explore interface. On the left, a tree view shows the site structure with 'GET /' selected. The main panel shows the 'Manual Explore' screen with a URL to explore set to 'http://localhost:3000/'. Below this, there are checkboxes for 'Enable HUD' and 'Explore your application' with buttons for 'Launch Browser' and 'Firefox'. A message states: 'You can also use browsers that you don't launch from ZAP, but will need to configure them to proxy through ZAP and to import the ZAP root CA certificate.'

At the bottom, the 'Alerts' pane shows a list of alerts. The first alert is 'SQL Injection (2)' with a severity of 'High'. The description reads: 'SQL injection may be possible.' The 'Other Info' section is empty. The 'Solution' section states: 'Do not trust client side input, even if there is client side validation in place. In general, never check all data on the server side.'

Remediation Steps:

Implement strict server-side input validation and output encoding.

Use secure frontend frameworks that automatically sanitize user input.

Apply Content Security Policy (CSP) protections to reduce XSS impact.

Finding 6: SQL Injection Vulnerability

Risk Level: High

Vulnerability Description: The application fails to properly validate and sanitize user-supplied input before processing database queries. This allows attackers to inject malicious SQL statements into application inputs, potentially manipulating backend database operations.

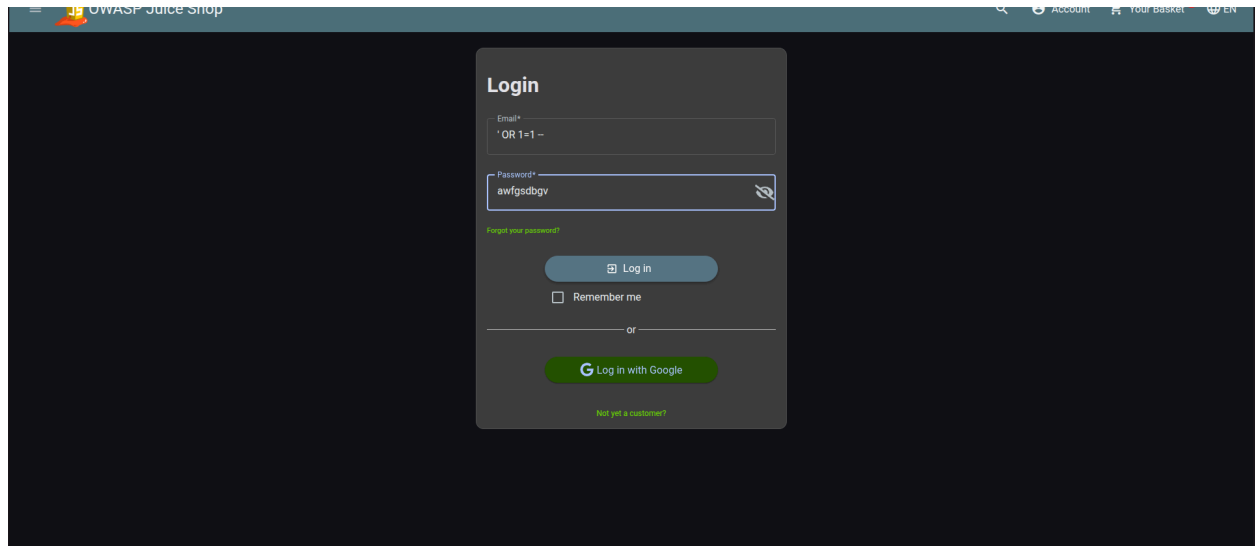
Business Impact: Successful exploitation of SQL Injection may allow attackers to bypass authentication mechanisms, access sensitive customer data, modify database contents, or completely compromise the backend database server.

Evidence: (OWASP ZAP Alert: SQL Injection)

Tested Parameter: email

Example Payload: ' OR '1'='1

Affected Endpoint: http://localhost:3000/rest/user/login



Remediation Steps:

Implement parameterized queries or prepared statements for all database interactions.

Avoid constructing SQL queries using direct string concatenation with user input.

Apply strict server-side input validation and sanitization.

Use ORM frameworks that support secure query handling mechanisms.

Implement least-privilege access controls for database accounts.

Finding 7: Sensitive Information Disclosure via Client-Side JavaScript

Risk Level: Medium

Vulnerability Description: During analysis using browser Developer Tools (DevTools), sensitive application information was identified within client-side JavaScript files and API responses. Internal API routes, debugging information, and backend-related details were exposed to unauthenticated users.

Business Impact: Exposed internal application details provide attackers with valuable reconnaissance data that can assist in identifying hidden endpoints, understanding application architecture, and planning targeted attacks against backend services.

Evidence: (*Browser DevTools → Sources Tab*)

Observed Files: [main.js](#) / [vendor.js](#)

Exposed Information: • *Internal API endpoint references* • *Backend route structures*

• *Debug-related console messages*

Remediation Steps:

Remove unnecessary debugging information from production builds.

Minimize exposure of internal application logic in frontend JavaScript files.

Implement proper API access controls and avoid exposing sensitive backend details to unauthenticated users.

Use code minification and secure production deployment practices.

Tools Used

Nmap

OWASP ZAP (Passive)

Browser DevTools

Canva (report design)

Risk Summary

The assessment identified multiple security weaknesses related to missing security headers, insecure cookie configurations, excessive information disclosure, weak client-side validation mechanisms, and injection vulnerabilities.

Several findings were categorized as Medium and High severity due to their potential impact on application confidentiality, integrity, and user security.

If exploited successfully, these vulnerabilities could lead to unauthorized access, data leakage, session hijacking, and backend compromise.

Conclusion

The NMAP , OWASP ZAP and Browser DevTools assessment successfully identified several common web application security vulnerabilities within the OWASP Juice Shop environment.

The findings demonstrate the importance of implementing secure coding practices, proper input validation, secure authentication mechanisms, and hardened server configurations.

Applying the recommended remediation measures will significantly improve the security posture of the application and reduce the overall attack surface.

—Thank You—